

MCSL-209 Data Structures and Algorithms Lab

Solved program bank · all 24 problems from 12 question sets, December 2024 to December 2025

Singly linked lists

P1. Write a program in 'C' that accepts two singly linked lists A and B as input. Now, print a singly linked list that consists only those elements which are not common to both A and B.

[20 marks · Dec 2024 Set-II Q1]

```

#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *next; };

void append(struct Node **head, struct Node **tail, int v) {
    struct Node *p = malloc(sizeof(struct Node));
    if (!p) { printf("Memory allocation failed\n"); exit(1); }
    p->data = v;
    p->next = NULL;
    if (*head == NULL) *head = *tail = p;
    else { (*tail)->next = p; *tail = p; }
}

struct Node *readList(char name, int n) {
    struct Node *head = NULL, *tail = NULL;
    int v, i;
    for (i = 1; i <= n; i++) {
        printf("%c[%d]: ", name, i);
        scanf("%d", &v);
        append(&head, &tail, v);
    }
    return head;
}

int present(struct Node *h, int v) {
    while (h) { if (h->data == v) return 1; h = h->next; }
    return 0;
}

void display(struct Node *h) {
    if (!h) { printf("(empty)\n"); return; }
    while (h) { printf("%d", h->data); if (h->next) printf(" -> "); h = h->next; }
    printf("\n");
}

void freeList(struct Node *h) {
    while (h) { struct Node *t = h->next; free(h); h = t; }
}

int main(void) {
    int na, nb;
    struct Node *A, *B, *C = NULL, *ct = NULL, *p;

    printf("Number of elements in list A: ");
    scanf("%d", &na);
    A = readList('A', na);
    printf("Number of elements in list B: ");
    scanf("%d", &nb);
    B = readList('B', nb);

    for (p = A; p; p = p->next)
        if (!present(B, p->data) && !present(C, p->data)) append(&C, &ct, p->data);
    for (p = B; p; p = p->next)
        if (!present(A, p->data) && !present(C, p->data)) append(&C, &ct, p->data);

    printf("List A: "); display(A);
    printf("List B: "); display(B);
    printf("Elements not common to both: "); display(C);

    freeList(A); freeList(B); freeList(C);
    return 0;
}

```

Sample run

```
Number of elements in list A: 5
A[1]: 1
A[2]: 2
A[3]: 3
A[4]: 4
A[5]: 5
Number of elements in list B: 4
B[1]: 4
B[2]: 5
B[3]: 6
B[4]: 7
List A: 1 -> 2 -> 3 -> 4 -> 5
List B: 4 -> 5 -> 6 -> 7
Elements not common to both: 1 -> 2 -> 3 -> 6 -> 7
```

P2. Write a program in 'C' to implement a singly linked list that stores list of integers provided as input. Sort and display the list in ascending order. [20 marks · Dec 2024 Set-III Q2]

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *next; };

void append(struct Node **head, int v) {
    struct Node *p = malloc(sizeof(struct Node)), *t;
    if (!p) { printf("Memory allocation failed\n"); exit(1); }
    p->data = v;
    p->next = NULL;
    if (*head == NULL) { *head = p; return; }
    for (t = *head; t->next; t = t->next) ;
    t->next = p;
}

void sortAscending(struct Node *head) {
    struct Node *i, *j;
    for (i = head; i; i = i->next)
        for (j = i->next; j; j = j->next)
            if (j->data < i->data) { int t = i->data; i->data = j->data; j->data = t; }
}

void display(struct Node *h) {
    if (!h) { printf("(empty)\n"); return; }
    while (h) { printf("%d", h->data); if (h->next) printf(" -> "); h = h->next; }
    printf("\n");
}

void freeList(struct Node *h) {
    while (h) { struct Node *t = h->next; free(h); h = t; }
}

int main(void) {
    int n, i, v;
    struct Node *head = NULL;

    printf("How many integers: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        printf("Element %d: ", i);
        scanf("%d", &v);
        append(&head, v);
    }

    printf("List before sorting: "); display(head);
    sortAscending(head);
    printf("List in ascending order: "); display(head);

    freeList(head);
    return 0;
}
```

Sample run

```
How many integers: 6
Element 1: 42
Element 2: 7
Element 3: 19
Element 4: 3
Element 5: 25
Element 6: 11
List before sorting: 42 -> 7 -> 19 -> 3 -> 25 -> 11
List in ascending order: 3 -> 7 -> 11 -> 19 -> 25 -> 42
```

P3. Write a program to create two singly linked lists to store two polynomials. Now, create and display a third linked list that contains the addition of the two polynomials.

[20 marks · Jun 2025 Set-I Q2]

```

#include <stdio.h>
#include <stdlib.h>

struct Term { int coeff, expo; struct Term *next; };

void append(struct Term **head, struct Term **tail, int c, int e) {
    struct Term *p = malloc(sizeof(struct Term));
    if (!p) { printf("Memory allocation failed\n"); exit(1); }
    p->coeff = c;
    p->expo = e;
    p->next = NULL;
    if (*head == NULL) *head = *tail = p;
    else { (*tail)->next = p; *tail = p; }
}

struct Term *readPoly(int n, char name) {
    struct Term *head = NULL, *tail = NULL;
    int i, c, e;
    for (i = 1; i <= n; i++) {
        printf("%c term %d (coefficient exponent): ", name, i);
        scanf("%d %d", &c, &e);
        append(&head, &tail, c, e);
    }
    return head;
}

struct Term *addPoly(struct Term *a, struct Term *b) {
    struct Term *head = NULL, *tail = NULL;
    while (a && b) {
        if (a->expo > b->expo) { append(&head, &tail, a->coeff, a->expo); a = a->next; }
        else if (a->expo < b->expo) { append(&head, &tail, b->coeff, b->expo); b = b->next; }
        else {
            if (a->coeff + b->coeff != 0) append(&head, &tail, a->coeff + b->coeff, a->expo);
            a = a->next; b = b->next;
        }
    }
    while (a) { append(&head, &tail, a->coeff, a->expo); a = a->next; }
    while (b) { append(&head, &tail, b->coeff, b->expo); b = b->next; }
    return head;
}

void display(struct Term *p) {
    if (!p) { printf("0\n"); return; }
    while (p) {
        printf("%dx^%d", p->coeff, p->expo);
        if (p->next) printf(" + ");
        p = p->next;
    }
    printf("\n");
}

void freeList(struct Term *p) {
    while (p) { struct Term *t = p->next; free(p); p = t; }
}

int main(void) {
    int n1, n2;
    struct Term *p1, *p2, *p3;

    printf("Number of terms in polynomial 1: ");
    scanf("%d", &n1);
    printf("Enter terms in descending order of exponent\n");
    p1 = readPoly(n1, 'P');

```

```

    printf("Number of terms in polynomial 2: ");
    scanf("%d", &n2);
    printf("Enter terms in descending order of exponent\n");
    p2 = readPoly(n2, 'Q');

    p3 = addPoly(p1, p2);
    printf("Polynomial 1: "); display(p1);
    printf("Polynomial 2: "); display(p2);
    printf("Sum          : "); display(p3);

    freeList(p1); freeList(p2); freeList(p3);
    return 0;
}

```

Sample run

```

Number of terms in polynomial 1: 3
Enter terms in descending order of exponent
P term 1 (coefficient exponent): 5 3
P term 2 (coefficient exponent): 4 2
P term 3 (coefficient exponent): 2 0
Number of terms in polynomial 2: 3
Enter terms in descending order of exponent
Q term 1 (coefficient exponent): 3 3
Q term 2 (coefficient exponent): 1 2
Q term 3 (coefficient exponent): 6 0
Polynomial 1: 5x^3 + 4x^2 + 2x^0
Polynomial 2: 3x^3 + 1x^2 + 6x^0
Sum          : 8x^3 + 5x^2 + 8x^0

```

P4. Write a program in 'C' language to create a singly linked list of integers as input. Sort the elements of the list in descending order. Now, input a new integer as input and insert it into the sorted linked list at the appropriate position. [20 marks · Jun 2025 Set-II Q1]

```

#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *next; };

struct Node *newNode(int v) {
    struct Node *p = malloc(sizeof(struct Node));
    if (!p) { printf("Memory allocation failed\n"); exit(1); }
    p->data = v;
    p->next = NULL;
    return p;
}

void append(struct Node **head, int v) {
    struct Node *p = newNode(v), *t;
    if (*head == NULL) { *head = p; return; }
    for (t = *head; t->next; t = t->next) ;
    t->next = p;
}

void sortDescending(struct Node *head) {
    struct Node *i, *j;
    for (i = head; i; i = i->next)
        for (j = i->next; j; j = j->next)
            if (j->data > i->data) { int t = i->data; i->data = j->data; j->data = t; }
}

void insertSortedDesc(struct Node **head, int v) {
    struct Node *p = newNode(v), *cur = *head, *prev = NULL;
    while (cur && cur->data > v) { prev = cur; cur = cur->next; }
    p->next = cur;
    if (prev == NULL) *head = p; else prev->next = p;
}

void display(struct Node *h) {
    if (!h) { printf("(empty)\n"); return; }
    while (h) { printf("%d", h->data); if (h->next) printf(" -> "); h = h->next; }
    printf("\n");
}

void freeList(struct Node *h) {
    while (h) { struct Node *t = h->next; free(h); h = t; }
}

int main(void) {
    int n, i, v;
    struct Node *head = NULL;

    printf("How many integers: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        printf("Element %d: ", i);
        scanf("%d", &v);
        append(&head, v);
    }

    printf("List as entered: "); display(head);
    sortDescending(head);
    printf("Sorted (descending): "); display(head);

    printf("New integer to insert: ");
    scanf("%d", &v);
    insertSortedDesc(&head, v);
}

```

```
printf("After insertion: "); display(head);

freeList(head);
return 0;
}
```

Sample run

```
How many integers: 5
Element 1: 12
Element 2: 45
Element 3: 7
Element 4: 30
Element 5: 19
List as entered: 12 -> 45 -> 7 -> 30 -> 19
Sorted (descending): 45 -> 30 -> 19 -> 12 -> 7
New integer to insert: 25
After insertion: 45 -> 30 -> 25 -> 19 -> 12 -> 7
```

P5. Write a program to 'C' language that create two singly linked list of integers. Now, create and print a singly linked list that consists of only those elements, which are common to both the linked lists. [20 marks · Jun 2025 Set-III Q2]

```

#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *next; };

void append(struct Node **head, struct Node **tail, int v) {
    struct Node *p = malloc(sizeof(struct Node));
    if (!p) { printf("Memory allocation failed\n"); exit(1); }
    p->data = v;
    p->next = NULL;
    if (*head == NULL) *head = *tail = p;
    else { (*tail)->next = p; *tail = p; }
}

struct Node *readList(char name, int n) {
    struct Node *head = NULL, *tail = NULL;
    int i, v;
    for (i = 1; i <= n; i++) {
        printf("%c[%d]: ", name, i);
        scanf("%d", &v);
        append(&head, &tail, v);
    }
    return head;
}

int present(struct Node *h, int v) {
    while (h) { if (h->data == v) return 1; h = h->next; }
    return 0;
}

void display(struct Node *h) {
    if (!h) { printf("(empty)\n"); return; }
    while (h) { printf("%d", h->data); if (h->next) printf(" -> "); h = h->next; }
    printf("\n");
}

void freeList(struct Node *h) {
    while (h) { struct Node *t = h->next; free(h); h = t; }
}

int main(void) {
    int n1, n2;
    struct Node *A, *B, *C = NULL, *ct = NULL, *p;

    printf("Number of elements in list 1: ");
    scanf("%d", &n1);
    A = readList('A', n1);
    printf("Number of elements in list 2: ");
    scanf("%d", &n2);
    B = readList('B', n2);

    for (p = A; p; p = p->next)
        if (present(B, p->data) && !present(C, p->data)) append(&C, &ct, p->data);

    printf("List 1: "); display(A);
    printf("List 2: "); display(B);
    printf("Common elements: "); display(C);

    freeList(A); freeList(B); freeList(C);
    return 0;
}

```

Sample run

```
Number of elements in list 1: 5
A[1]: 1
A[2]: 2
A[3]: 3
A[4]: 4
A[5]: 5
Number of elements in list 2: 4
B[1]: 4
B[2]: 2
B[3]: 9
B[4]: 5
List 1: 1 -> 2 -> 3 -> 4 -> 5
List 2: 4 -> 2 -> 9 -> 5
Common elements: 2 -> 4 -> 5
```

P6. Write a program to create a singly linked list. Now input two integers say 'm' and 'n'. The task is to traverse the linked list such that it skips 'm' nodes, then delete next 'n' nodes, and continue the task till the end of the linked list. Finally display the resultant linked list.

[20 marks · Dec 2025 Set-I Q2]

```

#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *next; };

void append(struct Node **head, struct Node **tail, int v) {
    struct Node *p = malloc(sizeof(struct Node));
    if (!p) { printf("Memory allocation failed\n"); exit(1); }
    p->data = v;
    p->next = NULL;
    if (*head == NULL) *head = *tail = p;
    else { (*tail)->next = p; *tail = p; }
}

void skipMdeleteN(struct Node **head, int m, int n) {
    struct Node *cur = *head, *prev = NULL, *tmp;
    int i;
    if (m < 0 || n < 0) { printf("m and n must not be negative\n"); return; }
    if (n == 0) return;
    while (cur) {
        for (i = 0; i < m && cur; i++) { prev = cur; cur = cur->next; }
        if (m == 0) {
            for (i = 0; i < n && cur; i++) { tmp = cur->next; free(cur); cur = tmp; }
            *head = cur;
            prev = NULL;
            continue;
        }
        for (i = 0; i < n && cur; i++) { tmp = cur->next; free(cur); cur = tmp; }
        if (prev) prev->next = cur;
    }
}

void display(struct Node *h) {
    if (!h) { printf("(empty)\n"); return; }
    while (h) { printf("%d", h->data); if (h->next) printf(" -> "); h = h->next; }
    printf("\n");
}

void freeList(struct Node *h) {
    while (h) { struct Node *t = h->next; free(h); h = t; }
}

int main(void) {
    int count, i, v, m, n;
    struct Node *head = NULL, *tail = NULL;

    printf("How many nodes: ");
    scanf("%d", &count);
    for (i = 1; i <= count; i++) {
        printf("Node %d: ", i);
        scanf("%d", &v);
        append(&head, &tail, v);
    }

    printf("Enter m (nodes to skip): ");
    scanf("%d", &m);
    printf("Enter n (nodes to delete): ");
    scanf("%d", &n);

    printf("List before: "); display(head);
    skipMdeleteN(&head, m, n);
    printf("List after : "); display(head);
}

```

```
    freeList(head);  
    return 0;  
}
```

Sample run

```
How many nodes: 10  
Node 1: 1  
Node 2: 2  
Node 3: 3  
Node 4: 4  
Node 5: 5  
Node 6: 6  
Node 7: 7  
Node 8: 8  
Node 9: 9  
Node 10: 10  
Enter m (nodes to skip): 2  
Enter n (nodes to delete): 3  
List before: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> 10  
List after : 1 -> 2 -> 6 -> 7
```

P7. Write a program in C to create a singly linked list to store a single character in each node of a string. Traverse the linked list and find whether the linked list stores a palindrome or not.

[20 marks · Dec 2025 Set-II Q1]

```
#include <stdio.h>
#include <stdlib.h>

struct Node { char ch; struct Node *next; };

void append(struct Node **head, struct Node **tail, char c) {
    struct Node *p = malloc(sizeof(struct Node));
    if (!p) { printf("Memory allocation failed\n"); exit(1); }
    p->ch = c;
    p->next = NULL;
    if (*head == NULL) *head = *tail = p;
    else { (*tail)->next = p; *tail = p; }
}

int countNodes(struct Node *h) {
    int c = 0;
    while (h) { c++; h = h->next; }
    return c;
}

int isPalindrome(struct Node *head) {
    int n = countNodes(head), i, j;
    struct Node *p, *q;
    for (i = 0, j = n - 1; i < j; i++, j--) {
        int k;
        for (p = head, k = 0; k < i; k++) p = p->next;
        for (q = head, k = 0; k < j; k++) q = q->next;
        if (p->ch != q->ch) return 0;
    }
    return 1;
}

void display(struct Node *h) {
    while (h) { printf("%c", h->ch); h = h->next; }
    printf("\n");
}

void freeList(struct Node *h) {
    while (h) { struct Node *t = h->next; free(h); h = t; }
}

int main(void) {
    struct Node *head = NULL, *tail = NULL;
    int c;

    printf("Enter a string: ");
    while ((c = getchar()) != '\n' && c != EOF)
        append(&head, &tail, (char)c);

    if (head == NULL) { printf("Empty string, nothing to check\n"); return 0; }

    printf("Linked list: "); display(head);
    if (isPalindrome(head)) printf("The linked list stores a palindrome\n");
    else printf("The linked list does not store a palindrome\n");

    freeList(head);
    return 0;
}
```

Sample run

```
Enter a string: madam
Linked list: madam
The linked list stores a palindrome

Second run:
Enter a string: abca
Linked list: abca
The linked list does not store a palindrome
```

P8. Write a program in 'C' to create a singly linked list. Now input an integer say 'K' and swap Kth node from the beginning with the Kth node from the end of the linked list.

[20 marks · Dec 2025 Set-III Q1]

```

#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *next; };

void append(struct Node **head, struct Node **tail, int v) {
    struct Node *p = malloc(sizeof(struct Node));
    if (!p) { printf("Memory allocation failed\n"); exit(1); }
    p->data = v;
    p->next = NULL;
    if (*head == NULL) *head = *tail = p;
    else { (*tail)->next = p; *tail = p; }
}

int countNodes(struct Node *h) {
    int c = 0;
    while (h) { c++; h = h->next; }
    return c;
}

void swapKth(struct Node **head, int k) {
    int n = countNodes(*head), i, j, c;
    struct Node *x, *y, *px = NULL, *py = NULL, *t;

    if (k < 1 || k > n) { printf("Invalid K: list has only %d node(s)\n", n); return; }
    i = k;
    j = n - k + 1;
    if (i == j) { printf("Both positions are the same node, nothing to swap\n"); return; }
    if (i > j) { c = i; i = j; j = c; }

    for (x = *head, c = 1; c < i; c++) { px = x; x = x->next; }
    for (y = *head, c = 1; c < j; c++) { py = y; y = y->next; }

    if (py == x) { /* the two nodes are adjacent */
        x->next = y->next;
        y->next = x;
        if (px) px->next = y; else *head = y;
    } else {
        if (px) px->next = y; else *head = y;
        py->next = x;
        t = x->next;
        x->next = y->next;
        y->next = t;
    }
}

void display(struct Node *h) {
    if (!h) { printf("(empty)\n"); return; }
    while (h) { printf("%d", h->data); if (h->next) printf(" -> "); h = h->next; }
    printf("\n");
}

void freeList(struct Node *h) {
    while (h) { struct Node *t = h->next; free(h); h = t; }
}

int main(void) {
    int n, i, v, k;
    struct Node *head = NULL, *tail = NULL;

    printf("How many nodes: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {

```

```

        printf("Node %d: ", i);
        scanf("%d", &v);
        append(&head, &tail, v);
    }
    printf("Enter K: ");
    scanf("%d", &k);

    printf("List before: "); display(head);
    swapKth(&head, k);
    printf("List after : "); display(head);

    freeList(head);
    return 0;
}

```

Sample run

```

How many nodes: 7
Node 1: 10
Node 2: 20
Node 3: 30
Node 4: 40
Node 5: 50
Node 6: 60
Node 7: 70
Enter K: 2
List before: 10 -> 20 -> 30 -> 40 -> 50 -> 60 -> 70
List after : 10 -> 60 -> 30 -> 40 -> 50 -> 20 -> 70

Second run (K out of range):
How many nodes: 5
Node 1: 1
Node 2: 2
Node 3: 3
Node 4: 4
Node 5: 5
Enter K: 9
List before: 1 -> 2 -> 3 -> 4 -> 5
Invalid K: list has only 5 node(s)
List after : 1 -> 2 -> 3 -> 4 -> 5

```

Stacks and expression conversion/evaluation

P1. Write a program in 'C' to convert a prefix expression to a postfix expression.

[20 marks · Dec 2024 Set-III Q1]

```

#include <stdio.h>

#define MAX 50

char stack[MAX][MAX];
int top = -1;

int isOperator(char c) {
    return c == '+' || c == '-' || c == '*' || c == '/' || c == '^';
}

void push(char *s) {
    int i;
    if (top == MAX - 1) { printf("Stack overflow\n"); return; }
    top++;
    for (i = 0; s[i] != '\0'; i++) stack[top][i] = s[i];
    stack[top][i] = '\0';
}

int pop(char *out) {
    int i;
    if (top < 0) return 0;
    for (i = 0; stack[top][i] != '\0'; i++) out[i] = stack[top][i];
    out[i] = '\0';
    top--;
    return 1;
}

int readLine(char *buf, int size) {
    int c, n = 0;
    while ((c = getchar()) != '\n' && c != EOF)
        if (n < size - 1) buf[n++] = (char)c;
    buf[n] = '\0';
    return n;
}

int main(void) {
    char prefix[MAX], a[MAX], b[MAX], res[MAX];
    int i, n, j, k;

    printf("Enter a prefix expression: ");
    n = readLine(prefix, MAX);

    for (i = n - 1; i >= 0; i--) {
        if (prefix[i] == ' ') continue;
        if (!isOperator(prefix[i])) {
            char one[2];
            one[0] = prefix[i];
            one[1] = '\0';
            push(one);
        } else {
            if (!pop(a) || !pop(b)) { printf("Invalid prefix expression\n"); return 1; }
            k = 0;
            for (j = 0; a[j] != '\0'; j++) res[k++] = a[j];
            for (j = 0; b[j] != '\0'; j++) res[k++] = b[j];
            res[k++] = prefix[i];
            res[k] = '\0';
            push(res);
        }
    }

    if (top != 0) { printf("Invalid prefix expression\n"); return 1; }
    printf("Postfix expression: %s\n", stack[0]);
}

```

```
    return 0;  
}
```

Sample run

```
Enter a prefix expression: *+ab-cd  
Postfix expression: ab+cd-*
```

P2. Write a program in 'C' to implement a stack using two queues. [20 marks · Dec 2024 Set-IV Q1]

```

#include <stdio.h>

#define MAX 50

struct Queue { int a[MAX]; int front, rear; };

void initQ(struct Queue *q) { q->front = 0; q->rear = -1; }
int isEmptyQ(struct Queue *q) { return q->front > q->rear; }
int sizeQ(struct Queue *q) { return q->rear - q->front + 1; }

int enqueue(struct Queue *q, int v) {
    if (q->rear == MAX - 1) { printf("Queue full\n"); return 0; }
    q->a[++q->rear] = v;
    return 1;
}

int dequeue(struct Queue *q, int *v) {
    if (isEmptyQ(q)) return 0;
    *v = q->a[q->front++];
    return 1;
}

struct Queue q1, q2;

void push(int v) {
    if (enqueue(&q1, v)) printf("Pushed %d\n", v);
}

int pop(int *v) {
    int x;
    if (isEmptyQ(&q1)) { printf("Stack underflow\n"); return 0; }
    while (sizeQ(&q1) > 1) { dequeue(&q1, &x); enqueue(&q2, x); }
    dequeue(&q1, v);
    initQ(&q1); /* q1 is now empty: reuse it as the spare queue */
    while (dequeue(&q2, &x)) enqueue(&q1, x);
    initQ(&q2);
    return 1;
}

void display(void) {
    int i;
    if (isEmptyQ(&q1)) { printf("Stack: (empty)\n"); return; }
    printf("Stack (bottom to top): ");
    for (i = q1.front; i <= q1.rear; i++) printf("%d ", q1.a[i]);
    printf("\n");
}

int main(void) {
    int choice, v;

    initQ(&q1);
    initQ(&q2);
    while (1) {
        printf("1.Push 2.Pop 3.Display 4.Exit : ");
        if (scanf("%d", &choice) != 1) break;
        if (choice == 1) {
            printf("Value: ");
            scanf("%d", &v);
            push(v);
            display();
        } else if (choice == 2) {
            if (pop(&v)) printf("Popped %d\n", v);
            display();
        }
    }
}

```

```

        } else if (choice == 3) {
            display();
        } else if (choice == 4) {
            break;
        } else {
            printf("Invalid choice\n");
        }
    }
    return 0;
}

```

Sample run

```

1.Push 2.Pop 3.Display 4.Exit : 1
Value: 10
Pushed 10
Stack (bottom to top): 10
1.Push 2.Pop 3.Display 4.Exit : 1
Value: 20
Pushed 20
Stack (bottom to top): 10 20
1.Push 2.Pop 3.Display 4.Exit : 1
Value: 30
Pushed 30
Stack (bottom to top): 10 20 30
1.Push 2.Pop 3.Display 4.Exit : 2
Popped 30
Stack (bottom to top): 10 20
1.Push 2.Pop 3.Display 4.Exit : 2
Popped 20
Stack (bottom to top): 10
1.Push 2.Pop 3.Display 4.Exit : 4

```

P3. Write a program in C to convert an infix expression into postfix expression. Use the appropriate data structure. [20 marks · Dec 2025 Set-II Q2]

```

#include <stdio.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char c) {
    if (top == MAX - 1) { printf("Stack overflow\n"); return; }
    stack[++top] = c;
}

char pop(void) {
    if (top < 0) return '\0';
    return stack[top--];
}

char peek(void) { return top < 0 ? '\0' : stack[top]; }

int precedence(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;
}

int isOperand(char c) {
    return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || (c >= '0' && c <= '9');
}

int readLine(char *buf, int size) {
    int c, n = 0;
    while ((c = getchar()) != '\n' && c != EOF)
        if (n < size - 1) buf[n++] = (char)c;
    buf[n] = '\0';
    return n;
}

int main(void) {
    char infix[MAX], postfix[MAX];
    int i, k = 0;

    printf("Enter an infix expression: ");
    readLine(infix, MAX);

    for (i = 0; infix[i] != '\0'; i++) {
        char c = infix[i];
        if (c == ' ') continue;
        if (isOperand(c)) postfix[k++] = c;
        else if (c == '(') push(c);
        else if (c == ')') {
            while (top >= 0 && peek() != '(') postfix[k++] = pop();
            if (top < 0) { printf("Unbalanced parentheses\n"); return 1; }
            pop();
        } else if (precedence(c) > 0) {
            while (top >= 0 && peek() != '(' && precedence(peek()) >= precedence(c))
                postfix[k++] = pop();
            push(c);
        } else {
            printf("Invalid character '%c'\n", c);
            return 1;
        }
    }
}

```

```
while (top >= 0) {
    char c = pop();
    if (c == '(') { printf("Unbalanced parentheses\n"); return 1; }
    postfix[k++] = c;
}
postfix[k] = '\0';

printf("Postfix expression: %s\n", postfix);
return 0;
}
```

Sample run

```
Enter an infix expression: a+b*c-(d/e+f)*g
Postfix expression: abc*+de/f+g*-
```

P4. Write a program in 'C' to convert an infix expression into prefix expression. Use appropriate data structure. [20 marks · Dec 2025 Set-III Q2]

```

#include <stdio.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char c) {
    if (top == MAX - 1) { printf("Stack overflow\n"); return; }
    stack[++top] = c;
}

char pop(void) {
    if (top < 0) return '\0';
    return stack[top--];
}

char peek(void) { return top < 0 ? '\0' : stack[top]; }

int precedence(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;
}

int isOperand(char c) {
    return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || (c >= '0' && c <= '9');
}

int myLen(char *s) {
    int n = 0;
    while (s[n] != '\0') n++;
    return n;
}

void reverseSwapBrackets(char *src, char *dst) {
    int n = myLen(src), i, k = 0;
    for (i = n - 1; i >= 0; i--) {
        if (src[i] == ' ') continue;
        if (src[i] == '(') dst[k++] = ')';
        else if (src[i] == ')') dst[k++] = '(';
        else dst[k++] = src[i];
    }
    dst[k] = '\0';
}

void reverse(char *src, char *dst) {
    int n = myLen(src), i, k = 0;
    for (i = n - 1; i >= 0; i--) dst[k++] = src[i];
    dst[k] = '\0';
}

int readLine(char *buf, int size) {
    int c, n = 0;
    while ((c = getchar()) != '\n' && c != EOF)
        if (n < size - 1) buf[n++] = (char)c;
    buf[n] = '\0';
    return n;
}

int main(void) {
    char infix[MAX], rev[MAX], tmp[MAX], prefix[MAX];

```

```

int i, k = 0;

printf("Enter an infix expression: ");
readLine(infix, MAX);
reverseSwapBrackets(infix, rev);

for (i = 0; rev[i] != '\0'; i++) {
    char c = rev[i];
    if (isOperand(c)) tmp[k++] = c;
    else if (c == '(') push(c);
    else if (c == ')') {
        while (top >= 0 && peek() != '(') tmp[k++] = pop();
        if (top < 0) { printf("Unbalanced parentheses\n"); return 1; }
        pop();
    } else if (precedence(c) > 0) {
        /* strictly greater, so equal precedence keeps right-to-left order */
        while (top >= 0 && peek() != '(' && precedence(peek()) > precedence(c))
            tmp[k++] = pop();
        push(c);
    } else {
        printf("Invalid character '%c'\n", c);
        return 1;
    }
}
while (top >= 0) {
    char c = pop();
    if (c == '(') { printf("Unbalanced parentheses\n"); return 1; }
    tmp[k++] = c;
}
tmp[k] = '\0';

reverse(tmp, prefix);
printf("Prefix expression: %s\n", prefix);
return 0;
}

```

Sample run

```

Enter an infix expression: (a-b/c)*(a/k-l)
Prefix expression: *-a/bc-/akl

```

P5. Write a program in 'C' to evaluate a post-fix expression. Use appropriate data structure for it. [20 marks · Dec 2025 Set-IV Q2]

```

#include <stdio.h>

#define MAX 100

double stack[MAX];
int top = -1;

int push(double v) {
    if (top == MAX - 1) { printf("Stack overflow\n"); return 0; }
    stack[++top] = v;
    return 1;
}

int pop(double *v) {
    if (top < 0) return 0;
    *v = stack[top--];
    return 1;
}

int readLine(char *buf, int size) {
    int c, n = 0;
    while ((c = getchar()) != '\n' && c != EOF)
        if (n < size - 1) buf[n++] = (char)c;
    buf[n] = '\0';
    return n;
}

int main(void) {
    char expr[MAX];
    int i = 0;
    double a, b, r;

    printf("Enter a postfix expression (tokens separated by spaces): ");
    readLine(expr, MAX);

    while (expr[i] != '\0') {
        if (expr[i] == ' ') { i++; continue; }
        if (expr[i] >= '0' && expr[i] <= '9') {
            double num = 0;
            while (expr[i] >= '0' && expr[i] <= '9') { num = num * 10 + (expr[i] - '0'); i++; }
            push(num);
            continue;
        }
        if (expr[i] == '+' || expr[i] == '-' || expr[i] == '*' || expr[i] == '/') {
            if (!pop(&b) || !pop(&a)) { /* b was pushed last, so a is the left operand */
                printf("Invalid expression: operator '%c' has fewer than two operands\n",
expr[i]);
                return 1;
            }
            if (expr[i] == '+') r = a + b;
            else if (expr[i] == '-') r = a - b;
            else if (expr[i] == '*') r = a * b;
            else {
                if (b == 0) { printf("Division by zero\n"); return 1; }
                r = a / b;
            }
            printf("%g %c %g = %g\n", a, expr[i], b, r);
            push(r);
            i++;
            continue;
        }
        printf("Invalid character '%c'\n", expr[i]);
        return 1;
    }
}

```

```

    }

    if (top != 0) { printf("Invalid expression\n"); return 1; }
    printf("Value of the expression = %g\n", stack[0]);
    return 0;
}

```

Sample run

```

Enter a postfix expression (tokens separated by spaces): 12 4 - 3 * 5 +
12 - 4 = 8
8 * 3 = 24
24 + 5 = 29
Value of the expression = 29

```

Strings

P1. Write a program in 'C' to concatenate two user input strings S1 and S2.

[20 marks · Dec 2024 Set-I Q1]

```

#include <stdio.h>

#define MAX 100

int readLine(char *buf, int size) {
    int c, n = 0;
    while ((c = getchar()) != '\n' && c != EOF)
        if (n < size - 1) buf[n++] = (char)c;
    buf[n] = '\0';
    return n;
}

int myLen(char *s) {
    int n = 0;
    while (s[n] != '\0') n++;
    return n;
}

int main(void) {
    char s1[2 * MAX], s2[MAX];
    int i, j;

    printf("Enter string S1: ");
    readLine(s1, MAX);
    printf("Enter string S2: ");
    readLine(s2, MAX);

    i = myLen(s1);
    if (i + myLen(s2) >= (int)sizeof(s1)) { printf("Destination too small\n"); return 1; }
    for (j = 0; s2[j] != '\0'; j++) s1[i + j] = s2[j];
    s1[i + j] = '\0';

    printf("Concatenated string: %s\n", s1);
    printf("Length: %d\n", myLen(s1));
    return 0;
}

```

Sample run

```
Enter string S1: Data
Enter string S2: Structures
Concatenated string: Data Structures
Length: 15
```

P2. Write a program in 'C' to reverse an input string. [20 marks · Dec 2024 Set-II Q2]

```
#include <stdio.h>

#define MAX 100

int readLine(char *buf, int size) {
    int c, n = 0;
    while ((c = getchar()) != '\n' && c != EOF)
        if (n < size - 1) buf[n++] = (char)c;
    buf[n] = '\0';
    return n;
}

int main(void) {
    char s[MAX];
    int n, i;

    printf("Enter a string: ");
    n = readLine(s, MAX);

    for (i = 0; i < n / 2; i++) {
        char t = s[i];
        s[i] = s[n - 1 - i];
        s[n - 1 - i] = t;
    }

    printf("Reversed string: %s\n", s);
    return 0;
}
```

Sample run

```
Enter a string: Linked List
Reversed string: tsil deknIL
```

P3. Write a program in 'C' language that read a string as input. Now reverse and print the string. Do not use any string input functions. [20 marks · Jun 2025 Set-II Q2]

```
#include <stdio.h>

#define MAX 100

int main(void) {
    char s[MAX], rev[MAX];
    int c, n = 0, i;

    printf("Enter a string: ");
    while ((c = getchar()) != '\n' && c != EOF)
        if (n < MAX - 1) s[n++] = (char)c;
    s[n] = '\0';

    for (i = 0; i < n; i++) rev[i] = s[n - 1 - i];
    rev[n] = '\0';

    printf("Original string: ");
    for (i = 0; s[i] != '\0'; i++) putchar(s[i]);
    printf("\nReversed string: ");
    for (i = 0; rev[i] != '\0'; i++) putchar(rev[i]);
    putchar('\n');
    return 0;
}
```

Sample run

```
Enter a string: IGNOU MCSL
Original string: IGNOU MCSL
Reversed string: LSCM UONGI
```

P4. Write a program in 'C' language that reads two strings as input and concatenate the second string after the first. Print the concatenated string. Do not use string inbuilt function for concatenation. [20 marks · Jun 2025 Set-III Q1]

```
#include <stdio.h>

#define MAX 100

int readLine(char *buf, int size) {
    int c, n = 0;
    while ((c = getchar()) != '\n' && c != EOF)
        if (n < size - 1) buf[n++] = (char)c;
    buf[n] = '\0';
    return n;
}

int main(void) {
    char s1[MAX], s2[MAX], result[2 * MAX];
    int i, k = 0;

    printf("Enter the first string: ");
    readLine(s1, MAX);
    printf("Enter the second string: ");
    readLine(s2, MAX);

    for (i = 0; s1[i] != '\0'; i++) result[k++] = s1[i];
    for (i = 0; s2[i] != '\0'; i++) result[k++] = s2[i];
    result[k] = '\0';

    printf("Concatenated string: %s\n", result);
    return 0;
}
```

Sample run

```
Enter the first string: Data Structures
Enter the second string:  and Algorithms
Concatenated string: Data Structures and Algorithms
```

Trees

P1. Write a program in 'C' for pre-order, post-order, and in-order traversal of a binary tree.

[20 marks · Dec 2024 Set-IV Q2]

```

#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *left, *right; };

struct Node *newNode(int v) {
    struct Node *p = malloc(sizeof(struct Node));
    if (!p) { printf("Memory allocation failed\n"); exit(1); }
    p->data = v;
    p->left = p->right = NULL;
    return p;
}

/* builds a binary tree level by level: each value read becomes the next
   vacant child in breadth-first order */
struct Node *buildTree(int n) {
    struct Node **q, *root, *child;
    int front = 0, rear = 0, i, v;

    if (n <= 0) return NULL;
    q = malloc(n * sizeof(struct Node *));
    if (!q) { printf("Memory allocation failed\n"); exit(1); }

    printf("Value 1 (root): ");
    scanf("%d", &v);
    root = newNode(v);
    q[rear++] = root;

    for (i = 2; i <= n; i++) {
        printf("Value %d: ", i);
        scanf("%d", &v);
        child = newNode(v);
        if (q[front]->left == NULL) q[front]->left = child;
        else { q[front]->right = child; front++; }
        q[rear++] = child;
    }
    free(q);
    return root;
}

void preorder(struct Node *r) {
    if (r == NULL) return;
    printf("%d ", r->data);
    preorder(r->left);
    preorder(r->right);
}

void inorder(struct Node *r) {
    if (r == NULL) return;
    inorder(r->left);
    printf("%d ", r->data);
    inorder(r->right);
}

void postorder(struct Node *r) {
    if (r == NULL) return;
    postorder(r->left);
    postorder(r->right);
    printf("%d ", r->data);
}

void freeTree(struct Node *r) {
    if (r == NULL) return;

```

```

    freeTree(r->left);
    freeTree(r->right);
    free(r);
}

int main(void) {
    int n;
    struct Node *root;

    printf("How many nodes: ");
    scanf("%d", &n);
    root = buildTree(n);
    if (root == NULL) { printf("Empty tree\n"); return 0; }

    printf("Pre-order : "); preorder(root); printf("\n");
    printf("In-order  : "); inorder(root);  printf("\n");
    printf("Post-order: "); postorder(root); printf("\n");

    freeTree(root);
    return 0;
}

```

Sample run

```

How many nodes: 7
Value 1 (root): 1
Value 2: 2
Value 3: 3
Value 4: 4
Value 5: 5
Value 6: 6
Value 7: 7
Pre-order : 1 2 4 5 3 6 7
In-order  : 4 2 5 1 6 3 7
Post-order: 4 5 2 6 7 3 1

```

P2. Write a program in 'C' language for the creation of a binary tree. Also, show the insertion and deletion operations on the tree created. [20 marks · Jun 2025 Set-I Q1]

```

#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *left, *right; };

struct Node *newNode(int v) {
    struct Node *p = malloc(sizeof(struct Node));
    if (!p) { printf("Memory allocation failed\n"); exit(1); }
    p->data = v;
    p->left = p->right = NULL;
    return p;
}

struct Node *insert(struct Node *root, int v) {
    if (root == NULL) return newNode(v);
    if (v < root->data) root->left = insert(root->left, v);
    else if (v > root->data) root->right = insert(root->right, v);
    else printf("%d already present, not inserted\n", v);
    return root;
}

struct Node *minNode(struct Node *r) {
    while (r->left) r = r->left;
    return r;
}

struct Node *deleteNode(struct Node *root, int v) {
    if (root == NULL) { printf("%d not found\n", v); return NULL; }
    if (v < root->data) root->left = deleteNode(root->left, v);
    else if (v > root->data) root->right = deleteNode(root->right, v);
    else {
        if (root->left == NULL) { /* leaf or only a right child */
            struct Node *r = root->right;
            free(root);
            return r;
        }
        if (root->right == NULL) { /* only a left child */
            struct Node *l = root->left;
            free(root);
            return l;
        }
        struct Node *s = minNode(root->right); /* two children: in-order successor */
        root->data = s->data;
        root->right = deleteNode(root->right, s->data);
    }
    return root;
}

void inorder(struct Node *r) {
    if (r == NULL) return;
    inorder(r->left);
    printf("%d ", r->data);
    inorder(r->right);
}

void preorder(struct Node *r) {
    if (r == NULL) return;
    printf("%d ", r->data);
    preorder(r->left);
    preorder(r->right);
}

void freeTree(struct Node *r) {

```

```

    if (r == NULL) return;
    freeTree(r->left);
    freeTree(r->right);
    free(r);
}

void show(struct Node *root) {
    printf("In-order : "); inorder(root); printf("\n");
    printf("Pre-order: "); preorder(root); printf("\n");
}

int main(void) {
    struct Node *root = NULL;
    int n, i, v, choice;

    printf("How many nodes to create: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        printf("Value %d: ", i);
        scanf("%d", &v);
        root = insert(root, v);
    }
    printf("Tree created.\n");
    show(root);

    while (1) {
        printf("1.Insert 2.Delete 3.Display 4.Exit : ");
        if (scanf("%d", &choice) != 1) break;
        if (choice == 1) {
            printf("Value to insert: ");
            scanf("%d", &v);
            root = insert(root, v);
            show(root);
        } else if (choice == 2) {
            printf("Value to delete: ");
            scanf("%d", &v);
            root = deleteNode(root, v);
            show(root);
        } else if (choice == 3) {
            show(root);
        } else if (choice == 4) {
            break;
        } else {
            printf("Invalid choice\n");
        }
    }

    freeTree(root);
    return 0;
}

```

Sample run

```
How many nodes to create: 7
Value 1: 50
Value 2: 30
Value 3: 70
Value 4: 20
Value 5: 40
Value 6: 60
Value 7: 80
Tree created.
In-order : 20 30 40 50 60 70 80
Pre-order: 50 30 20 40 70 60 80
1.Insert 2.Delete 3.Display 4.Exit : 1
Value to insert: 35
In-order : 20 30 35 40 50 60 70 80
Pre-order: 50 30 20 40 35 70 60 80
1.Insert 2.Delete 3.Display 4.Exit : 2
Value to delete: 30
In-order : 20 35 40 50 60 70 80
Pre-order: 50 35 20 40 70 60 80
1.Insert 2.Delete 3.Display 4.Exit : 2
Value to delete: 80
In-order : 20 35 40 50 60 70
Pre-order: 50 35 20 40 70 60
1.Insert 2.Delete 3.Display 4.Exit : 4
```

P3. Write a program in 'C' language to create a binary search tree. Display the binary search tree in in-order traversal. [20 marks · Jun 2025 Set-IV Q1]

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *left, *right; };

struct Node *insert(struct Node *root, int v) {
    if (root == NULL) {
        struct Node *p = malloc(sizeof(struct Node));
        if (!p) { printf("Memory allocation failed\n"); exit(1); }
        p->data = v;
        p->left = p->right = NULL;
        return p;
    }
    if (v < root->data) root->left = insert(root->left, v);
    else if (v > root->data) root->right = insert(root->right, v);
    else printf("%d is a duplicate, ignored\n", v);
    return root;
}

void inorder(struct Node *r) {
    if (r == NULL) return;
    inorder(r->left);
    printf("%d ", r->data);
    inorder(r->right);
}

void freeTree(struct Node *r) {
    if (r == NULL) return;
    freeTree(r->left);
    freeTree(r->right);
    free(r);
}

int main(void) {
    struct Node *root = NULL;
    int n, i, v;

    printf("How many values: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        printf("Value %d: ", i);
        scanf("%d", &v);
        root = insert(root, v);
    }

    if (root == NULL) { printf("Tree is empty\n"); return 0; }
    printf("Binary search tree in in-order traversal: ");
    inorder(root);
    printf("\n");

    freeTree(root);
    return 0;
}
```

Sample run

How many values: 7

Value 1: 55

Value 2: 20

Value 3: 70

Value 4: 10

Value 5: 35

Value 6: 65

Value 7: 90

Binary search tree in in-order traversal: 10 20 35 55 65 70 90

Queues

P1. Write a program in C to implement doubly ended queue using array. Show insertion and deletion operations on it. [20 marks · Dec 2025 Set-I Q1]

```

#include <stdio.h>

#define SIZE 5

int dq[SIZE];
int front = -1, rear = -1;

int isEmpty(void) { return front == -1; }
int isFull(void) { return (front == 0 && rear == SIZE - 1) || (front == rear + 1); }

void display(void) {
    int i;
    if (isEmpty()) { printf("Deque: (empty)\n"); return; }
    printf("Deque (front to rear): ");
    i = front;
    while (1) {
        printf("%d ", dq[i]);
        if (i == rear) break;
        i = (i + 1) % SIZE;
    }
    printf("\n");
}

void insertFront(int v) {
    if (isFull()) { printf("Deque is full\n"); return; }
    if (isEmpty()) { front = rear = 0; }
    else front = (front - 1 + SIZE) % SIZE;
    dq[front] = v;
    printf("Inserted %d at front\n", v);
    display();
}

void insertRear(int v) {
    if (isFull()) { printf("Deque is full\n"); return; }
    if (isEmpty()) { front = rear = 0; }
    else rear = (rear + 1) % SIZE;
    dq[rear] = v;
    printf("Inserted %d at rear\n", v);
    display();
}

void deleteFront(void) {
    if (isEmpty()) { printf("Deque is empty\n"); return; }
    printf("Deleted %d from front\n", dq[front]);
    if (front == rear) front = rear = -1;
    else front = (front + 1) % SIZE;
    display();
}

void deleteRear(void) {
    if (isEmpty()) { printf("Deque is empty\n"); return; }
    printf("Deleted %d from rear\n", dq[rear]);
    if (front == rear) front = rear = -1;
    else rear = (rear - 1 + SIZE) % SIZE;
    display();
}

int main(void) {
    int choice, v;

    while (1) {
        printf("1.InsertFront 2.InsertRear 3.DeleteFront 4.DeleteRear 5.Display 6.Exit : ");
        if (scanf("%d", &choice) != 1) break;
    }
}

```

```

        if (choice == 1 || choice == 2) {
            printf("Value: ");
            scanf("%d", &v);
            if (choice == 1) insertFront(v); else insertRear(v);
        } else if (choice == 3) deleteFront();
        else if (choice == 4) deleteRear();
        else if (choice == 5) display();
        else if (choice == 6) break;
        else printf("Invalid choice\n");
    }
    return 0;
}

```

Sample run

```

1.InsertFront 2.InsertRear 3.DeleteFront 4.DeleteRear 5.Display 6.Exit : 2
Value: 10
Inserted 10 at rear
Deque (front to rear): 10
1.InsertFront 2.InsertRear 3.DeleteFront 4.DeleteRear 5.Display 6.Exit : 2
Value: 20
Inserted 20 at rear
Deque (front to rear): 10 20
1.InsertFront 2.InsertRear 3.DeleteFront 4.DeleteRear 5.Display 6.Exit : 1
Value: 5
Inserted 5 at front
Deque (front to rear): 5 10 20
1.InsertFront 2.InsertRear 3.DeleteFront 4.DeleteRear 5.Display 6.Exit : 3
Deleted 5 from front
Deque (front to rear): 10 20
1.InsertFront 2.InsertRear 3.DeleteFront 4.DeleteRear 5.Display 6.Exit : 4
Deleted 20 from rear
Deque (front to rear): 10
1.InsertFront 2.InsertRear 3.DeleteFront 4.DeleteRear 5.Display 6.Exit : 6

```

Doubly linked lists

P1. Write a program in 'C' to create a doubly linked list. Now take an input say 'P' (a positive integer and smaller than the count of nodes in the doubly linked list created) and rotate the doubly linked list counter-clockwise by 'P' nodes. Display the resultant doubly linked list.

[20 marks · Dec 2025 Set-IV Q1]

```

#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *prev, *next; };

struct Node *head = NULL, *tail = NULL;

void append(int v) {
    struct Node *p = malloc(sizeof(struct Node));
    if (!p) { printf("Memory allocation failed\n"); exit(1); }
    p->data = v;
    p->next = NULL;
    p->prev = tail;
    if (tail) tail->next = p; else head = p;
    tail = p;
}

int countNodes(void) {
    int c = 0;
    struct Node *p = head;
    while (p) { c++; p = p->next; }
    return c;
}

void displayForward(void) {
    struct Node *p = head;
    if (!p) { printf("(empty)\n"); return; }
    while (p) { printf("%d", p->data); if (p->next) printf(" <-> "); p = p->next; }
    printf("\n");
}

void displayBackward(void) {
    struct Node *p = tail;
    if (!p) { printf("(empty)\n"); return; }
    while (p) { printf("%d", p->data); if (p->prev) printf(" <-> "); p = p->prev; }
    printf("\n");
}

void rotateCounterClockwise(int pp) {
    struct Node *pnode = head;
    int i;
    for (i = 1; i < pp; i++) pnode = pnode->next;
    /* pnode is the P-th node; the node after it becomes the new head */
    tail->next = head;
    head->prev = tail;
    head = pnode->next;
    head->prev = NULL;
    tail = pnode;
    tail->next = NULL;
}

void freeList(void) {
    struct Node *p = head, *t;
    while (p) { t = p->next; free(p); p = t; }
    head = tail = NULL;
}

int main(void) {
    int n, i, v, p;

    printf("How many nodes: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {

```

```

        printf("Node %d: ", i);
        scanf("%d", &v);
        append(v);
    }

    printf("List (forward) : "); displayForward();
    printf("List (backward): "); displayBackward();

    printf("Enter P: ");
    scanf("%d", &p);
    if (p <= 0 || p >= countNodes()) {
        printf("P must be positive and smaller than the number of nodes (%d)\n", countNodes());
        freeList();
        return 1;
    }

    rotateCounterClockwise(p);
    printf("After rotating counter-clockwise by %d node(s)\n", p);
    printf("List (forward) : "); displayForward();
    printf("List (backward): "); displayBackward();

    freeList();
    return 0;
}

```

Sample run

```

How many nodes: 6
Node 1: 10
Node 2: 20
Node 3: 30
Node 4: 40
Node 5: 50
Node 6: 60
List (forward) : 10 <-> 20 <-> 30 <-> 40 <-> 50 <-> 60
List (backward): 60 <-> 50 <-> 40 <-> 30 <-> 20 <-> 10
Enter P: 2
After rotating counter-clockwise by 2 node(s)
List (forward) : 30 <-> 40 <-> 50 <-> 60 <-> 10 <-> 20
List (backward): 20 <-> 10 <-> 60 <-> 50 <-> 40 <-> 30

```

Matrices

P1. Write a program in 'C' language that accepts two matrices as input and prints their products. [20 marks · Jun 2025 Set-IV Q2]

```
#include <stdio.h>

#define MAX 10

int main(void) {
    int a[MAX][MAX], b[MAX][MAX], c[MAX][MAX];
    int m, n, p, q, i, j, k;

    printf("Rows and columns of matrix A: ");
    scanf("%d %d", &m, &n);
    printf("Rows and columns of matrix B: ");
    scanf("%d %d", &p, &q);

    if (m <= 0 || n <= 0 || p <= 0 || q <= 0 || m > MAX || n > MAX || p > MAX || q > MAX) {
        printf("Dimensions must be between 1 and %d\n", MAX);
        return 1;
    }
    if (n != p) {
        printf("Multiplication not possible: columns of A (%d) != rows of B (%d)\n", n, p);
        return 1;
    }

    printf("Enter elements of A (%d x %d):\n", m, n);
    for (i = 0; i < m; i++)
        for (j = 0; j < n; j++) scanf("%d", &a[i][j]);

    printf("Enter elements of B (%d x %d):\n", p, q);
    for (i = 0; i < p; i++)
        for (j = 0; j < q; j++) scanf("%d", &b[i][j]);

    for (i = 0; i < m; i++)
        for (j = 0; j < q; j++) {
            c[i][j] = 0;
            for (k = 0; k < n; k++) c[i][j] += a[i][k] * b[k][j];
        }

    printf("Product (%d x %d):\n", m, q);
    for (i = 0; i < m; i++) {
        for (j = 0; j < q; j++) printf("%6d", c[i][j]);
        printf("\n");
    }
    return 0;
}
```

Sample run

```
Rows and columns of matrix A: 2 3
Rows and columns of matrix B: 3 2
Enter elements of A (2 x 3):
1 2 3
4 5 6
Enter elements of B (3 x 2):
7 8
9 10
11 12
Product (2 x 2):
    58    64
   139   154
```

Structures and records

P1. Write a program in 'C' language, which accepts student's details such as name, roll_number, address, aggregate marks secured by them. Rank the students according to the marks secured and display their details in the descending order of rank. [20 marks · Dec 2024 Set-I Q2]

```

#include <stdio.h>

#define MAXS 50

struct student {
    char name[40];
    char roll_number[20];
    char address[60];
    float marks;
};

void flushLine(void) {
    int c;
    while ((c = getchar()) != '\n' && c != EOF) ;
}

void readLine(char *buf, int size) {
    int c, n = 0;
    while ((c = getchar()) != '\n' && c != EOF)
        if (n < size - 1) buf[n++] = (char)c;
    buf[n] = '\0';
}

int main(void) {
    struct student s[MAXS], t;
    int n, i, j;

    printf("Number of students: ");
    scanf("%d", &n);
    flushLine();
    if (n <= 0 || n > MAXS) { printf("Count must be between 1 and %d\n", MAXS); return 1; }

    for (i = 0; i < n; i++) {
        printf("Student %d name: ", i + 1);
        readLine(s[i].name, sizeof s[i].name);
        printf("Roll number: ");
        readLine(s[i].roll_number, sizeof s[i].roll_number);
        printf("Address: ");
        readLine(s[i].address, sizeof s[i].address);
        printf("Aggregate marks: ");
        scanf("%f", &s[i].marks);
        flushLine();
    }

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - 1 - i; j++)
            if (s[j].marks < s[j + 1].marks) { t = s[j]; s[j] = s[j + 1]; s[j + 1] = t; }

    printf("\nRank   Name                               Roll      Marks   Address\n");
    for (i = 0; i < n; i++) {
        int rank = i + 1;
        if (i > 0 && s[i].marks == s[i - 1].marks) rank = i; /* equal marks share a rank */
        printf("%-5d %-20s %-10s %-7.2f %s\n", rank, s[i].name, s[i].roll_number,
            s[i].marks, s[i].address);
    }
    return 0;
}

```

Sample run

Number of students: 3
Student 1 name: Anita Rao
Roll number: 2301
Address: New Delhi
Aggregate marks: 78.5
Student 2 name: Vikram Singh
Roll number: 2302
Address: Pune
Aggregate marks: 91.25
Student 3 name: Meera Nair
Roll number: 2303
Address: Kochi
Aggregate marks: 84

Rank	Name	Roll	Marks	Address
1	Vikram Singh	2302	91.25	Pune
2	Meera Nair	2303	84.00	Kochi
3	Anita Rao	2301	78.50	New Delhi